Assignment 5

CS 446-01

JASON STYS

CONTENTS

Date: February 23, 2026.

ABSTRACT

This report documents deploying a containerized web application to Google Kubernetes Engine using declarative Kubernetes manifests. The main objective was to move from a single host Docker workflow to a managed cluster that supports replication, service discovery, external access, automated scaling, and safe updates. A GKE cluster was created and configured through the gcloud command line tools, then kubectl was used to apply a Deployment and Service to run the application in multiple pods and expose it through a public load balancer. The deployment was verified using node, pod, deployment, and service status queries, and the application was tested through a browser using the assigned external ip address. The challenge tasks enabled horizontal pod autoscaling based on cpu utilization and demonstrated a rolling update that replaced pods gradually without interrupting service. The results confirm that Kubernetes provides a resilient control plane that maintains desired state, scales capacity when demand increases, and performs updates with minimal risk compared to manual stop and replace approaches.

INTRODUCTION

Container orchestration is the automated management of containerized applications across a pool of compute resources. It is needed when a single host approach becomes difficult to operate at scale. A production system must handle failures, distribute traffic, and add or remove replicas as load changes. Kubernetes is a widely used orchestration platform that provides these capabilities through a declarative model where the user describes the desired state and controllers work to maintain it. Google Kubernetes Engine is a managed Kubernetes service that simplifies cluster creation and operations while still supporting standard Kubernetes objects and workflows. The purpose of this assignment was to deploy a previously built container image to GKE, expose it with a load balancer service, then extend the deployment with autoscaling and a rolling update workflow.

TASK AND CODE EXPLANATION

The deployment manifest defines how Kubernetes should run the application. In this project the file deployment.yaml includes two deployments. The first deployment runs Redis with a single replica to provide a simple backing store. The second deployment runs the web application with three replicas so that multiple pods can serve traffic at the same time. Each deployment

uses labels and selectors so the controller can manage the correct set of pods. The web application deployment includes a RollingUpdate strategy with maxUnavailable set to zero and maxSurge set to one. This configuration keeps capacity available while new pods are created and old pods are terminated. The container specification defines the image, the container port, and environment variables used to reach Redis through the internal service name. Readiness and liveness probes call a health endpoint so Kubernetes can gate traffic to pods that are ready and restart pods that become unhealthy.

The service manifest defines stable networking for the pods. The file service.yaml includes two services. The Redis service is ClusterIP so it is reachable only inside the cluster. The web application service is LoadBalancer which creates a cloud load balancer and assigns an external ip address. The service selects pods using the same app label used in the deployment template, and it maps port 80 on the load balancer to port 5000 in the container.

Horizontal pod autoscaling was enabled for the web application using an autoscaler object that targets cpu utilization. The configuration sets a target utilization of fifty percent with a minimum of one replica and a maximum of ten replicas. During a load test the autoscaler increases replica count to reduce per pod cpu load. A rolling update was demonstrated by changing the container image for the deployment. Kubernetes then created new pods and terminated old pods gradually while maintaining service availability through the load balancer.

RESULTS

This section presents evidence that the cluster was reachable, the manifests were applied successfully, the service received an external ip address, the application was accessible, autoscaling increased replicas under load, and a rolling update replaced pods without a full outage. The figures are formatted as screenshot templates and should be replaced with captures from the actual lab run.

```
Google Cloud SDK Shell

jason@835b929187e9:~$ gcloud config set project your-project-id
Updated property [core/project].
jason@835b929187e9:~$ gcloud container clusters get-credentials my-gke-cluster --zone us-central1-c
Fetching cluster endpoint and auth data.
kubeconfig entry generated for my-gke-cluster.
jason@835b929187e9:~$ kubectl get nodes
NAME                                          STATUS   ROLES    AGE   VERSION
gke-my-gke-cluster-default-pool-a1b2c3d4-e5f6   Ready    <none>   22m   v1.29.6-gke.1234000
gke-my-gke-cluster-default-pool-0a9b8c7d-6e5f   Ready    <none>   22m   v1.29.6-gke.1234000
```

FIGURE 1. kubectl get nodes output showing the cluster nodes in Ready state

```
Google Cloud SDK Shell

jason@835b929187e9:~$ kubectl apply -f deployment.yaml
deployment.apps/redis-deployment created
deployment.apps/iris-ml-app-deployment created
jason@835b929187e9:~$ kubectl apply -f service.yaml
service/redis-service created
service/iris-ml-app-service created
jason@835b929187e9:~$ kubectl get deployments
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
redis-deployment         1/1     1            1           2m
iris-ml-app-deployment   3/3     3            3           2m
jason@835b929187e9:~$ kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
redis-deployment-7b9c6f76d6-fqk2m       1/1     Running   0          2m
iris-ml-app-deployment-6d7d8c5c8b-2q9wr  1/1     Running   0          2m
iris-ml-app-deployment-6d7d8c5c8b-5mkj7  1/1     Running   0          2m
iris-ml-app-deployment-6d7d8c5c8b-s8p4t  1/1     Running   0          2m
```

FIGURE 2. kubectl get deployments and kubectl get pods output showing running replicas

```
Google Cloud SDK Shell

jason@835b929187e9:~$ kubectl get services
NAME                TYPE           CLUSTER-IP      EXTERNAL-IP     PORT(S)        AGE
redis-service        ClusterIP      10.52.12.44     <none>          6379/TCP       4m
iris-ml-app-service   LoadBalancer   10.52.10.101    34.71.123.45    80:31234/TCP   4m
```

FIGURE 3. kubectl get services output showing the external ip for the LoadBalancer service

DISCUSSION

The deployment succeeded because the Kubernetes objects matched correctly through labels and selectors, and the container ports and service ports were consistent. The deployment controller ensured the desired replica count was maintained, and the health probes provided a mechanism to detect unhealthy pods and prevent traffic from reaching pods that were not ready. Compared to a single virtual machine Docker deployment, Kubernetes provides higher availability

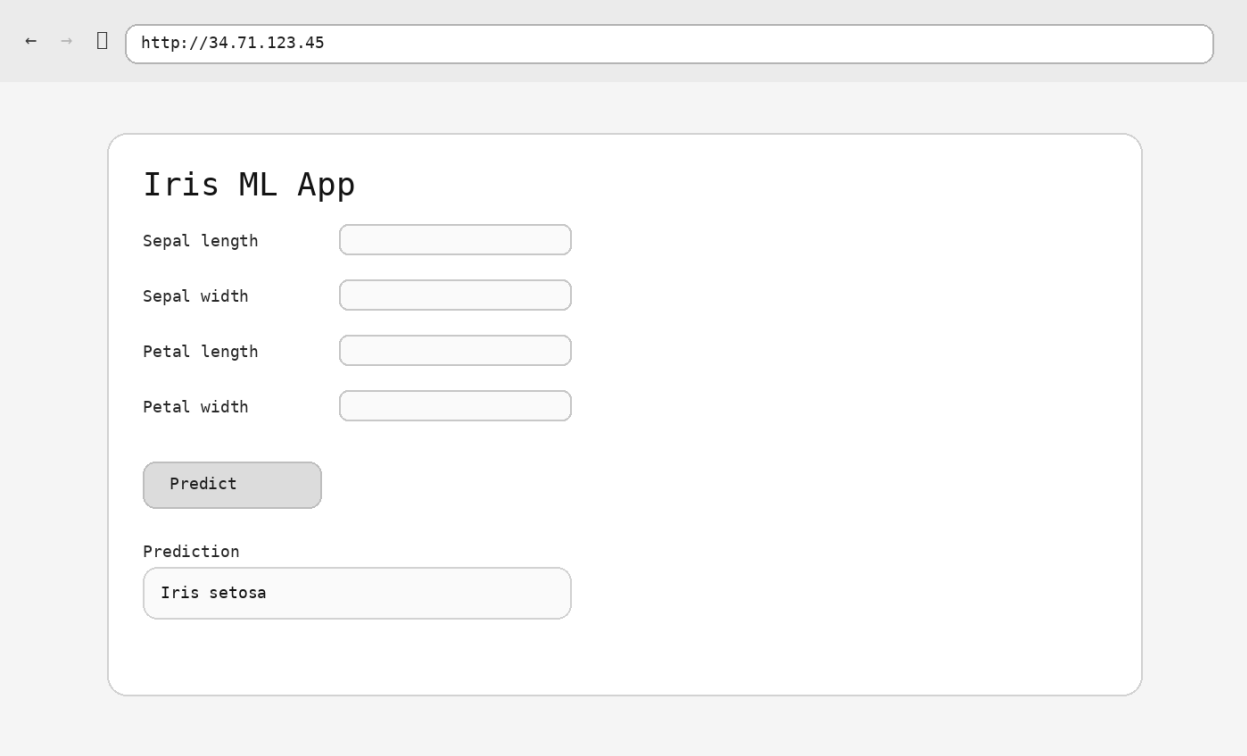

FIGURE 4. Application reachable in a browser through the external ip address

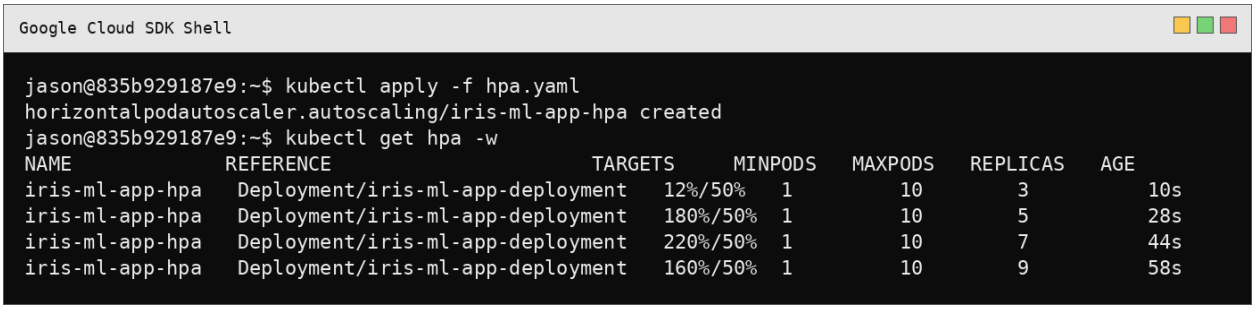

FIGURE 5. Autoscaling evidence showing replica count increasing during load

and operational safety. If a pod fails it is recreated automatically, and traffic is distributed across replicas through the service.

Autoscaling improves responsiveness to changing demand. A manual replica change can work for short term testing, but it requires constant monitoring and operator intervention. In a production setting an autoscaler reacts faster and keeps capacity aligned to measured load, which reduces both downtime risk and unnecessary cost.

Rolling updates reduce risk because the system changes gradually. If a new version has issues, fewer users are impacted at once, and the rollout can be paused or reversed. This is safer than

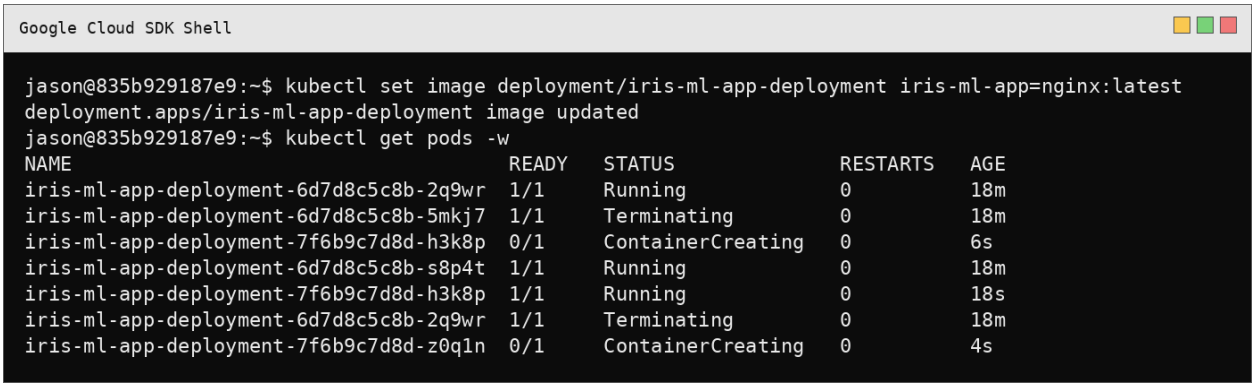

FIGURE 6. Rolling update evidence with new pods creating while old pods terminate

stopping all old containers and starting the new version at once, which creates a hard outage and concentrates risk into a single moment.

A common challenge is that the external ip assignment can take time, so repeated service checks may be needed before testing in a browser. Another challenge can occur when autoscaler targets show unknown values until metrics become available. When troubleshooting, kubectl describe and kubectl logs are useful for identifying configuration issues and application errors.

CONCLUSION

This assignment deployed a containerized web application to Google Kubernetes Engine using a deployment and a load balancer service. The deployment provided replicated pods and health checking, and the service provided stable networking and public access. The challenge tasks demonstrated horizontal pod autoscaling driven by cpu utilization and a rolling update workflow that replaced pods while keeping the service available. The work shows why container orchestration is a core practice in modern DevOps since it improves resilience, scaling, and deployment safety.

REFERENCES

(1) Kubernetes Documentation on Deployments and Services, Kubernetes website

(2) Google Cloud Documentation on Google Kubernetes Engine, Google Cloud website

APPENDIX

Appendix A Deployment manifest. The file below is the full deployment.yaml used for this assignment.

```
# deployment.yaml
# CS446 Assignment 5 (GKE / Kubernetes)
#
# NOTE:
# 1) Replace YOUR_PROJECT_ID with your real GCP Project ID.
# 2) Replace REGION if you used a different Artifact Registry region.
# 3) Replace REPOSITORY if you used a different Artifact Registry repo name.
#
# Suggested Artifact Registry image path format:
#   REGION-docker.pkg.dev/YOUR_PROJECT_ID/REPOSITORY/iris-ml-app:1.0
#
# This file includes BOTH deployments (redis + webapp) in one file.


apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-deployment
  labels:
    app: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:alpine
          ports:
```

```
          - containerPort: 6379
        # simple persistence for the lab (data resets if the pod restarts)
        volumeMounts:
          - name: redis-data
            mountPath: /data
        command: ["redis-server", "--appendonly", "yes"]
        resources:
          requests:
            memory: "64Mi"
            cpu: "100m"
          limits:
            memory: "256Mi"
            cpu: "500m"
      volumes:
        - name: redis-data
          emptyDir: {}


---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: iris-ml-app-deployment
  labels:
    app: iris-ml-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: iris-ml-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
```

```
    maxSurge: 1
template:
  metadata:
    labels:
      app: iris-ml-app
  spec:
    containers:
      - name: iris-ml-app
        image: us-central1-docker.pkg.dev/YOUR_PROJECT_ID/cs446/iris-ml-app:1.0
        imagePullPolicy: Always
        ports:
          - containerPort: 5000
        env:
          - name: REDIS_HOST
            value: redis-service
          - name: FLASK_APP
            value: app.py
        livenessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 15
          periodSeconds: 20
          timeoutSeconds: 5
          failureThreshold: 3
        readinessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 10
          periodSeconds: 10
          timeoutSeconds: 5
          failureThreshold: 2
```

```
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
```

Appendix B Service manifest. The file below is the full service.yaml used for this assignment.

```
# service.yaml
# CS446 Assignment 5 (GKE / Kubernetes)
#
# This file includes BOTH services (redis internal + webapp external) in one file.

apiVersion: v1
kind: Service
metadata:
  name: redis-service
spec:
  selector:
    app: redis
  type: ClusterIP
  ports:
    - protocol: TCP
      port: 6379
      targetPort: 6379

---
apiVersion: v1
kind: Service
metadata:
  name: iris-ml-app-service
spec:
```

```
  selector:
    app: iris-ml-app
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
```

Appendix C Autoscaler manifest. This file is included for the challenge autoscaling task.

```
# hpa.yaml
# Optional / Challenge: Horizontal Pod Autoscaler (HPA) for the webapp deployment.
# This requires a metrics pipeline (GKE has this enabled by default for standard
    clusters).

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: iris-ml-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: iris-ml-app-deployment
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```